Planning and Reinforcement Learning in AI Systems

(PARLAI)

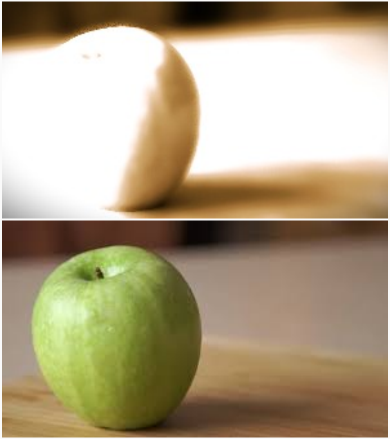
Lesson 5a - Planning Under Partial Observability

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Agency under Partial Observability

Partial Observability



In classical and stochastic planning, we assumed the agent has access to the true state.

More precisely, the agent observes all relevant features needed for decision-making.

This assumption is called full observability.

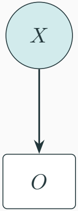
In reality,

- the true state is often **hidden**
- the agent receives **partial and noisy observations**

Instead of observing the current state directly, the agent receives O_t generated from the hidden state via the sensor model

$$\Pr(O_t \mid X_t)$$

The sensor model captures sensor noise, limited sensing range or resolution, ambiguity (multiple states producing similar observations), etc.

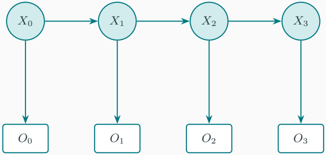


Partial Observability Over Time

$t = 3$

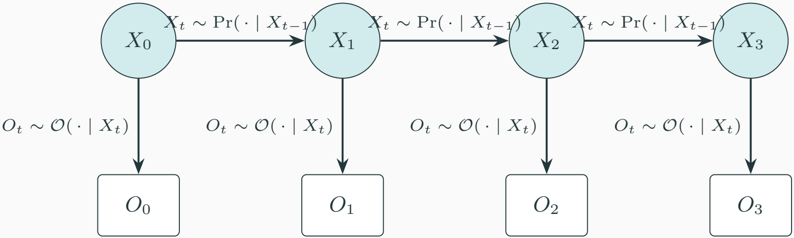
$t = 15$

Partial Observability Over Time



- When the environment is dynamic, over time, the agent observes only the **observation sequence** $O_{0:T} = (O_0, O_1, \ldots, O_T)$, while the **state sequence** $X_{0:T} = (X_0, X_1, \ldots, X_T)$ remains unobserved.
- At each time step, we now have **two** coupled processes:
  - **State dynamics:** $X_t \sim \Pr(\cdot \mid X_{t-1})$
  - **Observation generation:** $O_t \sim \mathcal{O}(\cdot \mid X_t)$

Partial Observability Over Time



Partial Observability Over Time

- Modeling assumptions:
  - Markov dynamics:

$$\Pr(X_{t+1} \mid X_{0:t}) = \Pr(X_{t+1} \mid X_t)$$

  - Memoryless observations:

$$\Pr(O_t \mid X_{0:t}, O_{0:t-1}) = \Pr(O_t \mid X_t)$$

- Under these assumptions, the following conditional independencies hold:

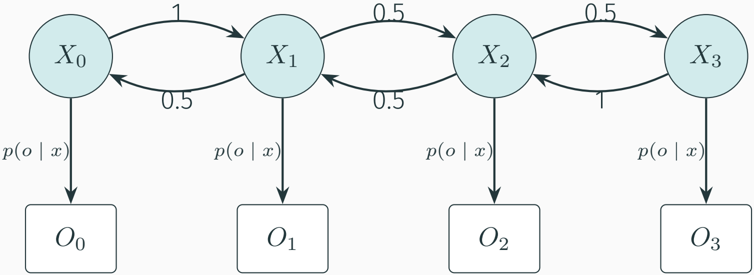
$$X_{t+1} \perp\!\!\!\perp X_{0:t-1} \mid X_t, \qquad O_t \perp\!\!\!\perp (O_{0:t-1}, X_{0:t-1}) \mid X_t$$

- Therefore, the joint distribution factors as:

$$\Pr(X_{0:T}, O_{0:T}) = \Pr(X_0) \prod_{t=0}^{T-1} \Pr(X_{t+1} \mid X_t) \prod_{t=0}^{T} \Pr(O_t \mid X_t)$$

This structure defines a Hidden Markov Model (HMM).

Hidden Markov Model (HMM)

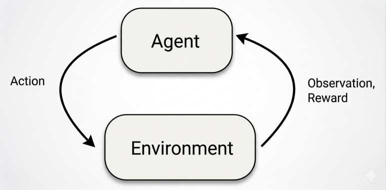


How do we formulate and optimize agency?

Agency under partial observability

Recall that the environment-agent interactions give rise to a sequence or trajectory: $s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, \ldots$

From the point of view of the partially informed agent, we have a **history**: $o_0, a_0, r_1, o_1, a_1, r_2, o_2, a_2, r_3, \ldots$



The agent needs to maintain its belief based on the current (possibly null) observation.

Reminder: Partially Observable Markov Decision Process (POMDP)

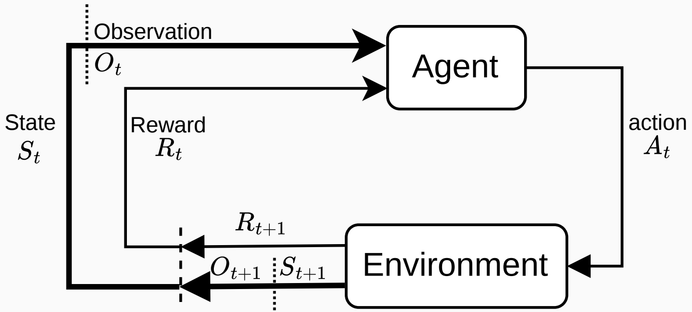
A **Partially Observable Markov Decision Process**(POMDP) is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \Omega, \mathcal{O}, \beta_0 \rangle$ where

- $\mathcal{S}$, $\mathcal{A}$, $\mathcal{P}$, $\mathcal{R}$ and γ are as for an MDP.
- Ω is a set of observations (observation tokens),
- $\mathcal{O}$ is a sensor function specifying the conditional observation probabilities $\mathcal{O}^o_{s,a} = \mathcal{P}[O_{t+1} = o | S_t = s, A_t = a]$ of receiving observation token $o \in \Omega$ in state s after applying action a [1].
- β_0 the initial belief: a probability distribution over the states such that $\beta_0(s)$ stands for the probability of s being the true initial state.

Markov property still holds

[1] alternatively: $\mathcal{O}^o_s = \mathcal{P}[O_t = o | S_t = s]$

Stochastic Partially Observable Environments



Information State

An **information state** I_t is a summary of all past actions and observations that is sufficient for optimal decision-making.

> What does it mean to be **sufficient** and **optimal** under partial knowledge?

Information State

Information State

Information State

An information state I_t is a summary of all past actions and observations that is sufficient for optimal decision-making.

Intuition: Knowing I_t is as informative as knowing the full history h_t.

Requirements:

- **Sufficiency:** I_t retains all history information relevant for predicting future states and rewards.
- **Markov Property:** $I_{t+1} = f(I_t, a_t, o_{t+1})$, i.e., no need to store the full history.
- **Reward/Observation Predictiveness:** $\mathbb{E}[r_t \mid h_t, a_t] = \mathbb{E}[r_t \mid I_t, a_t]$.

Belief $\beta_t(x)$ is a canonical information state.

Definition: Information State for a Reward POMDP

An **information state** I_t is a *sufficient statistic* of the interaction history $h_t = (o_{1:t}, a_{1:t-1}, r_{1:t})$ such that:

IS1) Sufficient for reward prediction:

$$\mathbb{E}[r_t \mid h_t, a_t] = \mathbb{E}[r_t \mid I_t = \sigma_t(h_t), a_t] \qquad \forall a_t$$

IS2) Sufficient to predict itself:

$$P[I_{t+1} \mid h_t, a_t] = P[I_{t+1} \mid I_t = \sigma_t(h_t), a_t] \qquad \forall a_t$$

Definition by Subramanian et al. 2022.

Definition: Information State for a [Reward] POMDP (cntd.)

The condition IS2 can be replaced by the stronger conditions:

IS2a) Recursive evolution: There exist functions $(\phi_t)_{t\in[T]}$ such that

$$I_{t+1} = \phi_t(o_{t+1}, I_t, a_t) \qquad \forall t \in [T-1]$$

IS2b) Sufficient for observation prediction:

$$P[o_{t+1} \mid h_t, a_t] = P[o_{t+1} \mid I_t = \sigma_t(h_t), a_t] \qquad \forall a_t$$

- The most common information state is the belief.
- A belief is a probability distribution over the hidden state:

$$\beta_t(x) = P(X_t = x \mid h_t).$$

Reminder: Bayes Filter

Algorithm 1 Bayes Filter

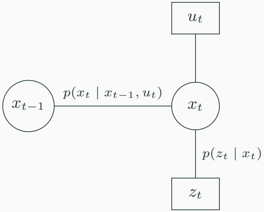
Require: $\beta(x_{t-1}), u_t, z_t$

for all x_t **do**

$\quad \overline{\beta}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, \beta(x_{t-1})\, dx$

$\quad \beta(x_t) = \eta\, p(z_t \mid x_t)\, \overline{\beta}(x_t)$

end for

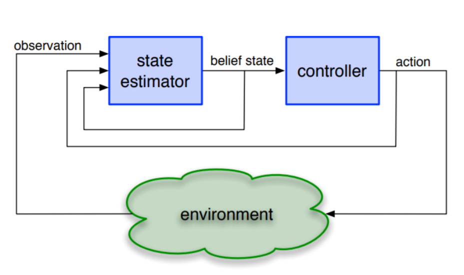


Planning Under Partial Information

Core challenge: The agent must act without knowing the true state, while [its] actions affect both the world *and* its knowledge.

Two key challenges when planning in belief space:

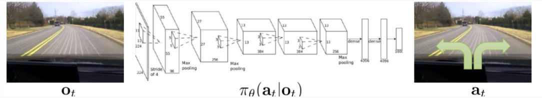
- **Belief tracking** - what is the state of the world ?
- **Policy computation** - what is the best action to perform ?



Pineau, Nicholas and Thrun. "A hierarchical approach to POMDP planning and execution." 2001. https://www.cs.mcgill.ca/ jpineau/files/jpineau-icml01.pdf

Can we use a **Markov Decision Process**(MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ to account for partially observable environments ?

Sometimes, yes.



Sometimes, an MDP is not enough.

Main Approaches

We will consider three main approaches:

- Replanning
- Conformant
- Contingent

Running example: Two identical boxes; only one contains a tool needed to complete the task.

- Plan under a single hypothesized state - often MAP (Maximum A Posteriori).
- Execute, observe, and replan when assumptions are violated.
- Simple and scalable, but reactive to uncertainty.

Running Example:

Planning Under Partial Knowledge: Replanning

- Plan under a single hypothesized state - often MAP (Maximum A Posteriori).
- Execute, observe, and replan when assumptions are violated.
- Simple and scalable, but reactive to uncertainty.

Running Example:

- Assume a single hypothesis (e.g., tool is in Box A).
- Plan: open Box A $\rightarrow$ pick-up tool.
- If observation contradicts assumption, replan for Box B.
- Fast and simple, but failures occur before recovery.

- Plan to succeed for *all* possible initial states.
- No sensing actions during execution.
- Provides strong guarantees but is often overly conservative.

Running Example:

- Plan to succeed for *all* possible initial states.
- No sensing actions during execution.
- Provides strong guarantees but is often overly conservative.

Running Example:

- Plan must succeed whether the tool is in Box A or Box B.
- Plan: open Box A, pick-up tool from Box A, place tool in Box B, pick-up tool from Box B.
- No sensing required, but unnecessary actions are taken.

- Explicitly reasons about sensing and observations.
- Produces branching plans conditioned on outcomes.
- Handles uncertainty proactively, but is computationally demanding.

Running Example

- Explicitly reasons about sensing and observations.
- Produces branching plans conditioned on outcomes.
- Handles uncertainty proactively, but is computationally demanding.

Running Example

- Plan includes sensing and branching.
- Plan: open Box A; sense tool; if tool found pick-up tool; else open Box B ; pick-up tool.
- Explicitly reasons about possible observations.

Relation to belief-space planning:

- Replanning corresponds to committing to a single state.
- Conformant planning treats belief as a set of possibilities.
- Contingent planning reasons over belief updates and observations.

Belief-space planning generalizes all three by optimizing decisions directly over uncertainty.

Planning Under Partial Knowledge: Three Approaches

Relation to belief-space planning:

- Replanning corresponds to committing to a single state.
- Conformant planning treats belief as a set of possibilities.
- Contingent planning reasons over belief updates and observations.

Belief-space planning generalizes all three by optimizing decisions directly over uncertainty.

POMDP formulation:

- States:
- Actions:
- Observations:
- Sensor Function:
- Belief:

Policy over belief:

Belief update after open(A):

POMDP formulation:

- **States:** $\{s_A, s_B\}$ (tool in Box A or Box B)
- **Actions:** open(A), open(B), pick-up(tool)
- **Observations:** tool-found, tool-not-found
- **Sensor Function:** noisy sensing for tool
- **Belief:** $\beta(s_A) = \beta(s_B) = 0.5$

Policy over belief:

Belief update after open(A):

POMDP formulation:

- **States:** $\{s_A, s_B\}$ (tool in Box A or Box B)
- **Actions:** open(A), open(B), pick-up(tool)
- **Observations:** tool-found, tool-not-found
- **Sensor Function:** noisy sensing for tool
- **Belief:** $\beta(s_A) = \beta(s_B) = 0.5$

Policy over belief:

$$\pi(\beta) = \begin{cases} \text{open(A)} & \text{if } \beta(s_A) \geq 0.5 \\ \text{open(B)} & \text{otherwise} \end{cases}$$

Belief update after open(A):

POMDP formulation:

- **States:** $\{s_A, s_B\}$ (tool in Box A or Box B)
- **Actions:** open(A), open(B), pick-up(tool)
- **Observations:** tool-found, tool-not-found
- **Sensor Function:** noisy sensing for tool
- **Belief:** $\beta(s_A) = \beta(s_B) = 0.5$

Policy over belief:

$$\pi(\beta) = \begin{cases} \text{open(A)} & \text{if } \beta(s_A) \geq 0.5 \\ \text{open(B)} & \text{otherwise} \end{cases}$$

Belief update after open(A):

$$\beta'(s_A) = \begin{cases} 1 & \text{if tool found} \\ 0 & \text{if not} \end{cases}$$

POMDP used to optimize *expected cost* and *information gain*

Conformant Planning

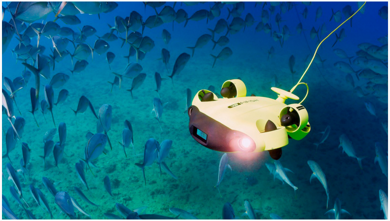
Planning under uncertainty without sensing

- Uncertain initial state
- Deterministic actions
- No observations during execution

			T

Motivation

- Initial conditions may be partially unknown
- Sensing may be unavailable or expensive
- Limited or no ability to plan (policy needs to be set before deployment)



Goal: Find a plan that works *regardless* of the true initial state.

A **conformant planning problem** is a tuple:

$$P = \langle F, A, I, G \rangle$$

where:

- F – fluents
- A – deterministic actions
- I – set of possible initial states
- G – goal condition

Requirement:

A plan must achieve G for all $s \in I$.

Logical Belief View

Belief:

$$\beta \subseteq 2^F$$

set of states consistent with current information.

Belief:

$$\beta \subseteq 2^F$$

set of states consistent with current information.

Initial belief:

$$\beta_0 = I$$

Belief:

$$\beta \subseteq 2^F$$

set of states consistent with current information.

Initial belief:

$$\beta_0 = I$$

Action progression:

Belief:

$$\beta \subseteq 2^F$$

set of states consistent with current information.

Initial belief:

$$\beta_0 = I$$

Action progression:

$$\beta_{t+1} = \{s' \mid \exists s \in \beta_t, s' = \mathcal{P}(s, a_t)\}$$

Belief:

$$\beta \subseteq 2^F$$

set of states consistent with current information.

Initial belief:

$$\beta_0 = I$$

Action progression:

$$\beta_{t+1} = \{s' \mid \exists s \in \beta_t, s' = \mathcal{P}(s, a_t)\}$$

Goal test:

Belief:

$$\beta \subseteq 2^F$$

set of states consistent with current information.

Initial belief:

$$\beta_0 = I$$

Action progression:

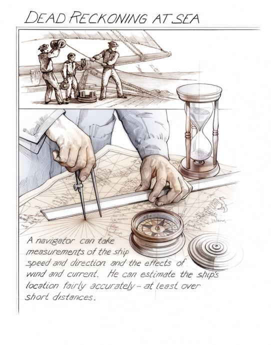
$$\beta_{t+1} = \{s' \mid \exists s \in \beta_t, s' = \mathcal{P}(s, a_t)\}$$

Goal test:

$$\forall s \in \beta_t,\ s \models G$$

Challenges

- Belief states grow exponentially
- No observations to reduce uncertainty
- Actions must be safe in all possible states (worlds)



Conformant Planning as Search in Belief Space

Key Idea 1

Search is performed over **belief space** rather than state space (each node represents a belief).

Key Idea 2

If actions have preconditions, an action is applicable only if it is applicable for **all** states in the belief.

Benefits? Limitations?

Conformant Planning as Search in Belief Space

Search Graph:

- Nodes: belief states $\beta \in \mathcal{B}$
- Edges: actions $a \in \mathcal{A}$

Algorithm Outline

- Start from initial belief β_0
- Expand all applicable actions
- Generate successor beliefs
- Track visited beliefs to avoid loops

Perform any state space search over the belief space

Let $\mathcal{F}$ be a set of Boolean features.

Number of states: $|\mathcal{S}| = 2^{|\mathcal{F}|}$

Since a belief state is a set of possible states, $\mathcal{B} = 2^{\mathcal{S}}$

and therefore $|\mathcal{B}| = 2^{|\mathcal{S}|} = 2^{2^{|\mathcal{F}|}}$

Belief-space planning is **doubly exponential** in the number of Boolean features.

A common approximation is to represent each feature as

$$\{\text{True}, \text{False}, \text{Unknown}\},$$

yielding $3^{|\mathcal{F}|}$ possible belief descriptions.

Solution Approach: Knowledge-Level Compilation

Key idea:

- Compile uncertainty away by reasoning at the **knowledge level**.
- Replace belief states with **Knowledge-level representation:** For every $p \in F$
  - Kp: if p is known true
  - $K\neg p$: if p is known false
  - the absence of both $(\neg Kp \wedge \neg K\neg p) \Rightarrow p$ is unknown
- Actions are associated with preconditions and effects.
- Reduce conformant planning to **classical planning**.

Solution Approach: Knowledge-Level Compilation

Action compilation:

- Original precondition $p \;\Rightarrow\;$ compiled precondition Kp
- Original effect $q \;\Rightarrow\;$ compiled effect Kq
- Knowledge is removed if an action may falsify it in some world (i.e., $s \in \beta$)

Initial state compilation:

- Kp if p is true in *all* initial states
- $K\neg p$ if p is false in *all* initial states
- Otherwise: unknown

Goal compilation: $G \;\Rightarrow\; KG$

Goal must be *known* to hold.

$$\forall p_i \in G, p_1 \wedge \cdots \wedge p_n \quad \Rightarrow \quad Kp_1 \wedge \cdots \wedge Kp_n$$

Contingent Planning

Problem Setting

- Like previously:
  - initial state may be *partially known*
  - agent can *sense* during execution
- Here: policy (instead of a plan) that may *branch* based on observations

Contingent Plans as AND–OR Trees

Plan Structure

- **OR nodes:** agent chooses an action
- **AND nodes:** environment reveals an observation

Execution

- Execute action
- Observe outcome
- Follow the corresponding branch

Correctness

All root-to-leaf paths must reach a goal state.

Contingent Plans as AND–OR Trees

Belief-Based Semantics

Agent maintains a belief $\beta \subseteq \mathcal{S}$ over possible states.

Policy Definition

A contingent plan defines a policy:

$$\pi : \mathcal{B} \to \mathcal{A}$$

where Ω is the set of possible observations.

Goal Condition

A belief β is goal-reaching iff:

$$\forall s \in \beta,\ s \models G$$

Contingent Plans as AND–OR Trees

Pros and cons:

- **+** Explicitly reasons over contingencies
- **+** Produces complete conditional plans
- **–** Exponential blow-up in observation branches

AND–OR graphs provide a clean semantics for contingent plans, but scalability motivates replanning and belief-based alternatives.

Planning on POMDPs

Key Differences

- **Contingent planning:**

  non -deterministic actions and observations

  - Finite branching
- **POMDPs:**
  - Probabilistic transitions and observations
  - Policies over continuous belief space

Planning Algorithms for POMDPs

Setting:

- State is not directly observable
- Agent maintains a **belief** $\beta(s)$
- Planning is performed in **belief space**

Key challenge:

- Belief space is **continuous and high-dimensional**
- Planning must trade off:
  - Exploitation (reward)
  - Exploration / information gathering

Main algorithmic question: How do we compute or approximate an optimal policy over beliefs?

Three main approaches to POMDP Planning

1. Exact Planning
   - Value Iteration, Policy Iteration (Sondik)
   - Requires computing value function over beliefs
   - Optimal but intractable
2. Approximate Offline Planning
   - PBVI, Perseus, HSVI, SARSOP
   - Sample reachable beliefs
   - Strong empirical performance
3. Online Planning
   - POMCP, POMCPOW, DESPOT
   - Monte-Carlo tree search in belief space
   - **Scales to large problems**

Exact POMDP methos

Belief update (Bayes filter):

$$\beta'(s') = \eta \, \mathcal{O}(o \mid s', a) \sum_{s} \mathcal{P}(s' \mid s, a) \beta(s)$$

- Belief β is a probability distribution over states
- Belief space $\mathcal{B}$ is **continuous**
- Planning becomes an MDP over beliefs

How do we compute an optimal policy $\pi(\beta)$ over $\mathcal{B}$?

Value Function over Beliefs

Bellman optimality equation:

$$V^*(\beta) = \max_{a \in \mathcal{A}} \left[\mathcal{R}(\beta, a) + \gamma \sum_{o \in \Omega} \mathcal{P}(o \mid \beta, a) \, V(\beta^{a,o}) \right]$$

- $\mathcal{R}(\beta, a) = \sum_s \beta(s) \mathcal{R}(s, a)$
- $\beta^{a,o}$ = updated belief after (a, o)

Structure of the Value Function

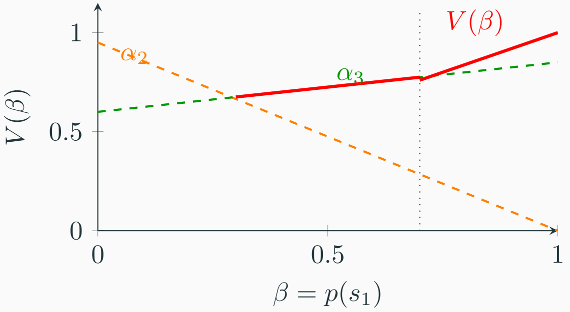
Theorem (Sondik, 1971):

- The optimal finite-horizon POMDP value function is:
  - **Piecewise-linear**
  - **Convex**
  - Over the belief simplex

Representation:

$$V(\beta) = \max_{\alpha \in \Gamma} \ \alpha^\top \beta$$

- Each α-vector corresponds to a conditional plan
- Value iteration constructs Γ iteratively



Interpretation:

- Each dashed line is an α-vector (a conditional plan)
- The red curve is the upper envelope $V(\beta) = \max_{\alpha} \alpha \cdot \beta$
- Belief space is partitioned into regions with different optimal plans

Exact Methods for Solving POMDPs

Goal: Compute the optimal value function

$$V(\beta) = \max_{\alpha \in \Gamma} \alpha \cdot \beta$$

exactly, over the continuous belief space.

Core idea:

- Represent $V(\beta)$ using a finite set of α-vectors
- Iteratively update this set via dynamic programming

Exact Methods for Solving POMDPs

Main exact methods:

- Value Iteration (Sondik)
  - Bellman backup over beliefs
  - Computes all relevant α-vectors
- Policy Iteration
  - Alternates between policy evaluation and improvement
  - Less common due to computational cost

Properties:

- Exponential complexity in $|S|$, $|O|$, and horizon
- Practical only for small POMDPs

Approximate Methods for Solving POMDPs

Exact methods are theoretically elegant but do not scale.
Approximation trades optimality for tractability

Approximate Methods for Solving POMDPs*

1) Point-Based Value Iteration (PBVI family)

- Approximate $V(b)$ on a finite belief set $\mathcal{B}$
- Maintain a reduced set of α-vectors

Representative methods:

- PBVI, Perseus
- HSVI (Heuristic Search Value Iteration)
- SARSOP (smart belief sampling)

2) Determinization-Based Approximations

- Simplify the POMDP into an easier planning problem
- Ignore or approximate information-gathering value

Examples:

- QMDP, FIB
- Most-likely-state planning

Conclusion

Evaluation

- **Classical considerations:** completeness, optimality, time complexity, space complexity, anytime guarantees
- **Modern considerations:** robustness, scalability, adaptivity, generality, interpretability, energy efficiency

Where do the methods we have seen stand?

Real-world AI systems rarely have complete and accurate knowledge of their environment.

Core challenges:

How should an agent act when it is uncertain about the world?

Three major approaches emerge:

- **Conformant planning:** Commit to a plan that succeeds despite uncertainty.
- **Contingent planning:** Construct a policy that adapts based on future observations.
- **Replanning:** Plan with current knowledge, then revise the plan as new information becomes available.

These ideas continue to appear throughout modern AI: robotics, reinforcement learning, autonomous driving, decision making under uncertainty, robust and adaptive AI systems.

Limitation of model-based planning:

- Accurate transition and observation models are hard to specify
- Environment dynamics may change over time
- Planning in large belief spaces remains expensive

Learning-based perspective:

- Learn policies or value functions directly from data
- Replace explicit belief updates with learned internal state
- Adapt online to new dynamics and sensing conditions
- Trade explicit models for adaptability and scale.

How to best combine both approaches?